

Assignment No: 2

Course Title: CS-426L Parallel and Distributing Computing

Object-Oriented Messaging, Data Structures, Multithreading and Concurrency in Java

SUBMITTED BY:

Student ID: 247198
Semester: 5 Semester
Section: BS CYS F 24 B

SUBMITTED TO:

Instructor Name: Mr. Qaiser Ali

Date of Submission: 10-Oct-2026

GitHub Repository: https://github.com/zyx981742/PDC-Assignmnet-2.git

AIR UNIVERSITY KHARIAN CAMPUS

DEPARTMENT OF COMPUTER SCIENCE

QUESTION NO: 1

Financial Trading Event Dispatcher Using Inheritance and Polymorphism

Design a four-file Java architecture for a Complex Event Processing (CEP) engine. TradeEvent is the superclass with protected senderId, timestamp and basePrice fields, a parameterized constructor and a baseline dispatch() method. FIXOrderEvent and ReutersQuoteEvent extend it and override dispatch(). TradingEngineApp stores the subclass objects in superclass references and calls dispatch() on each one to demonstrate runtime polymorphism.

File 1: TradeEvent.java

- **senderId, timestamp, basePrice**: protected fields of type String, long and double.
- **Constructor**: initializes all three protected fields.
- **dispatch() method**: logs the generic event details (baseline behaviour).

Main concept: Superclass holding the common state that every market event needs.

Java Program:

J TradeEvent.java

```java
public class TradeEvent {

    protected String senderId;
    protected long timestamp;
    protected double basePrice;

    public TradeEvent(String senderId, long timestamp, double basePrice) {
        this.senderId = senderId;
        this.timestamp = timestamp;
        this.basePrice = basePrice;
    }

    public void dispatch() {
        System.out.println("Generic Trade Event");
        System.out.println("Sender ID  : " + senderId);
        System.out.println("Timestamp  : " + timestamp);
        System.out.println("Base Price : " + basePrice);
    }
}
```

File 2: FIXOrderEvent.java

- **Extends TradeEvent**: inherits senderId, timestamp and basePrice.
- **clOrdId**: new String field for the FIX client order ID (a quantity field is added for the share count).
- **Constructor**: calls super(senderId, timestamp, basePrice) and then stores its own fields.
- **Overridden dispatch()**: formats a stock purchase request and applies the 0.15% exchange transaction fee.

Main concept: Inheritance with super() and method overriding.

Java Program:

J FIXOrderEvent.java

```
public class FIXOrderEvent extends TradeEvent {

    private static final double EXCHANGE_FEE_RATE = 0.0015; // 0.15%

    private String clOrdId;
    private int quantity;

    public FIXOrderEvent(String senderId, long timestamp, double basePrice,
                         String clOrdId, int quantity) {
        super(senderId, timestamp, basePrice);
        this.clOrdId = clOrdId;
        this.quantity = quantity;
    }

    @Override
    public void dispatch() {
        double orderValue = basePrice * quantity;
        double fee = orderValue * EXCHANGE_FEE_RATE;
        double totalCost = orderValue + fee;

        System.out.println("FIX Stock Purchase Order");
        System.out.println("Sender ID        : " + senderId);
        System.out.println("Timestamp        : " + timestamp);
        System.out.println("ClOrdID          : " + clOrdId);
        System.out.println("Quantity         : " + quantity + " shares");
        System.out.printf("Price per share  : %.2f%n", basePrice);
        System.out.printf("Order value      : %.2f%n", orderValue);
        System.out.printf("Exchange fee 0.15%%: %.2f%n", fee);
        System.out.printf("Total cost       : %.2f%n", totalCost);
        System.out.println("Status           : ORDER EXECUTED");
    }
}
```

File 3: ReutersQuoteEvent.java

- **Extends TradeEvent**: reuses the common event fields.
- **bidPrice, askPrice**: the two quote fields.
- **Overridden dispatch()**: prints the bid-ask spread, mid price and spread percentage.

Main concept: The same dispatch() call produces completely different market analytics.

Java Program:

J ReutersQuoteEvent.java

```
public class ReutersQuoteEvent extends TradeEvent {

    private double bidPrice;
```

```
4         private double askPrice;
5
6         public ReutersQuoteEvent(String senderId, long timestamp, double basePrice,
7                                  double bidPrice, double askPrice) {
8             super(senderId, timestamp, basePrice);
9             this.bidPrice = bidPrice;
10            this.askPrice = askPrice;
11        }
12
13        @Override
14        public void dispatch() {
15            double spread = askPrice - bidPrice;
16            double midPrice = (askPrice + bidPrice) / 2.0;
17            double spreadPercent = (spread / midPrice) * 100.0;
18
19            System.out.println("Reuters Real-Time Quote");
20            System.out.println("Sender ID     : " + senderId);
21            System.out.println("Timestamp     : " + timestamp);
22            System.out.printf("Reference     : %.2f%n", basePrice);
23            System.out.printf("Bid Price     : %.2f%n", bidPrice);
24            System.out.printf("Ask Price     : %.2f%n", askPrice);
25            System.out.printf("Bid-Ask Spread: %.2f%n", spread);
26            System.out.printf("Mid Price     : %.3f%n", midPrice);
27            System.out.printf("Spread %%      : %.4f%%%n", spreadPercent);
28        }
29    }
```

File 4: TradingEngineApp.java

- Creates FIXOrderEvent, ReutersQuoteEvent and TradeEvent objects.
- Stores every object in a variable of the superclass type TradeEvent.
- Calls dispatch() on each reference and prints the output to the console.

Main concept: Runtime polymorphism. The reference type is TradeEvent, but the JVM selects the overridden dispatch() of the actual object at run time.

Java Program:

J TradingEngineApp.java

```
1     public class TradingEngineApp {
2
3         public static void main(String[] args) {
4
5             // Subclass objects stored in SUPERCLASS reference variables
6             TradeEvent event1 = new FIXOrderEvent(
7                     "FIX-GATEWAY-01", 1761300000000L, 185.50, "ORD-90001", 100);
8
9             TradeEvent event2 = new ReutersQuoteEvent(
10                    "REUTERS-FEED-07", 1761300000450L, 185.55, 185.48, 185.62);
11
12            TradeEvent event3 = new TradeEvent(
13                    "GENERIC-SOURCE", 1761300000900L, 100.00);
```

```
14
15          TradeEvent[] events = { event1, event2, event3 };
16
17          System.out.println("========================================");
18          System.out.println("   COMPLEX EVENT PROCESSING (CEP) ENGINE");
19          System.out.println("========================================");
20
21          for (TradeEvent event : events) {
22              // Runtime polymorphism: JVM picks the overridden dispatch()
23              event.dispatch();
24              System.out.println("----------------------------------------");
25          }
26
27          System.out.println("      ALL EVENTS DISPATCHED SUCCESSFULLY");
28          System.out.println("========================================");
29      }
30  }
```

Program OutPut:

```
> java TradingEngineApp
========================================
   COMPLEX EVENT PROCESSING (CEP) ENGINE
========================================
FIX Stock Purchase Order
Sender ID        : FIX-GATEWAY-01
Timestamp        : 1761300000000
ClOrdID          : ORD-90001
Quantity         : 100 shares
Price per share  : 185.50
Order value      : 18550.00
Exchange fee 0.15%: 27.83
Total cost       : 18577.83
Status           : ORDER EXECUTED
----------------------------------------
Reuters Real-Time Quote
Sender ID     : REUTERS-FEED-07
Timestamp     : 1761300000450
Reference     : 185.55
Bid Price     : 185.48
Ask Price     : 185.62
Bid-Ask Spread: 0.14
Mid Price     : 185.550
Spread %      : 0.0755%
----------------------------------------
Generic Trade Event
Sender ID  : GENERIC-SOURCE
Timestamp  : 1761300000900
Base Price : 100.0
----------------------------------------
      ALL EVENTS DISPATCHED SUCCESSFULLY
========================================
```

Explanation

For the FIX order, order value = 185.50 × 100 = 18,550.00. The 0.15% exchange fee is 18,550.00 × 0.0015 = 27.825 (printed as 27.83), giving a total cost of 18,577.83. For the Reuters quote, the spread is ask − bid = 185.62 − 185.48 = 0.14.

QUESTION NO: 2

Protocol Packet Verification Using a Custom Stack and Queue

Build an array-backed stack (TransactionStack.java) and an array-backed queue (RequestQueue.java) without any built-in collection classes. PacketSymmetryAuditor.java ingests a packet payload, pushes every character into the stack while enqueueing it into the queue, then pops and dequeues together. Because a stack returns characters in reverse (LIFO) and a queue in original (FIFO) order, the packet is symmetric (a palindrome) only if every pair matches.

File 1: TransactionStack.java

- **char[] stack, top, capacity**: fixed-capacity storage with a top pointer starting at -1.
- push(char c), pop(), peek() and isEmpty() implemented at low level with overflow and underflow checks.

Main concept: LIFO: the last character pushed is the first one popped.

Java Program:

J TransactionStack.java

```java
public class TransactionStack {

    private char[] stack;
    private int top;
    private int capacity;

    public TransactionStack(int capacity) {
        this.capacity = capacity;
        stack = new char[capacity];
        top = -1;
    }

    public void push(char c) {
        if (top == capacity - 1) {
            System.out.println("Stack overflow!");
            return;
        }
        stack[++top] = c;
    }

    public char pop() {
        if (isEmpty()) {
            System.out.println("Stack underflow!");
            return '\0';
        }
        return stack[top--];
    }

    public char peek() {
        if (isEmpty()) {
            return '\0';
        }
```

```
33            return stack[top];
34        }
35
36        public boolean isEmpty() {
37            return top == -1;
38        }
39    }
```

File 2: RequestQueue.java

- **char[] queue, front, rear, size**: circular array so freed slots are reused.
- enqueue(char c), dequeue() and isEmpty() with overflow and underflow checks.

Main concept: FIFO: the first character enqueued is the first one dequeued.

Java Program:

J RequestQueue.java

```
1     public class RequestQueue {
2
3         private char[] queue;
4         private int front;
5         private int rear;
6         private int size;
7         private int capacity;
8
9         public RequestQueue(int capacity) {
10            this.capacity = capacity;
11            queue = new char[capacity];
12            front = 0;
13            rear = -1;
14            size = 0;
15        }
16
17        public void enqueue(char c) {
18            if (size == capacity) {
19                System.out.println("Queue overflow!");
20                return;
21            }
22            rear = (rear + 1) % capacity;
23            queue[rear] = c;
24            size++;
25        }
26
27        public char dequeue() {
28            if (isEmpty()) {
29                System.out.println("Queue underflow!");
30                return '\0';
31            }
32            char c = queue[front];
33            front = (front + 1) % capacity;
34            size--;
35            return c;
```

```
36        }
37
38        public boolean isEmpty() {
39            return size == 0;
40        }
41    }
```

File 3: PacketSymmetryAuditor.java

- ❖ verifySymmetry() pushes and enqueues every character of the payload.
- ❖ Pops and dequeues step by step and compares each pair (MATCH / MISMATCH).
- ❖ main() tests several packets and prints the verified terminal log.

Main concept: Comparing a reversed stream (stack) with the original stream (queue) detects palindrome signatures.

Java Program:

J PacketSymmetryAuditor.java

```
1     public class PacketSymmetryAuditor {
2
3         // Returns true when the payload reads the same forwards and backwards
4         public static boolean verifySymmetry(String payload) {
5
6             TransactionStack stack = new TransactionStack(payload.length());
7             RequestQueue queue = new RequestQueue(payload.length());
8
9             System.out.println("Ingesting packet : " + payload);
10
11            // Push into the LIFO stack while enqueueing into the FIFO queue
12            for (int i = 0; i < payload.length(); i++) {
13                char c = payload.charAt(i);
14                stack.push(c);
15                queue.enqueue(c);
16            }
17
18            boolean symmetric = true;
19            int position = 1;
20
21            // Pop (reverse order) and dequeue (original order) together
22            while (!stack.isEmpty() && !queue.isEmpty()) {
23                char fromStack = stack.pop();
24                char fromQueue = queue.dequeue();
25
26                String result = (fromStack == fromQueue) ? "MATCH" : "MISMATCH";
27                System.out.println("  Step " + position + ": pop = '" + fromStack
28                        + "'  dequeue = '" + fromQueue + "'  -> " + result);
29
30                if (fromStack != fromQueue) {
31                    symmetric = false;
32                }
33                position++;
```

```
            }
            return symmetric;
        }

        public static void main(String[] args) {

            String[] packets = { "A1B2B1A", "level", "X9Y8Z7", "racecar" };

            System.out.println("========================================");
            System.out.println("      PACKET SYMMETRY VERIFICATION LOG");
            System.out.println("========================================");

            for (String packet : packets) {
                boolean symmetric = verifySymmetry(packet);
                System.out.println("Result : " + packet + " -> "
                        + (symmetric ? "SYMMETRIC (Palindrome signature detected)"
                                     : "NOT SYMMETRIC"));
                System.out.println("----------------------------------------");
            }

            System.out.println("          AUDIT COMPLETE");
            System.out.println("========================================");
        }
    }
```

Program OutPut:

```
> java PacketSymmetryAuditor
========================================
      PACKET SYMMETRY VERIFICATION LOG
========================================
Ingesting packet : A1B2B1A
  Step 1: pop = 'A'  dequeue = 'A'  -> MATCH
  Step 2: pop = '1'  dequeue = '1'  -> MATCH
  Step 3: pop = 'B'  dequeue = 'B'  -> MATCH
  Step 4: pop = '2'  dequeue = '2'  -> MATCH
  Step 5: pop = 'B'  dequeue = 'B'  -> MATCH
  Step 6: pop = '1'  dequeue = '1'  -> MATCH
  Step 7: pop = 'A'  dequeue = 'A'  -> MATCH
Result : A1B2B1A -> SYMMETRIC (Palindrome signature detected)
----------------------------------------
Ingesting packet : level
  Step 1: pop = 'l'  dequeue = 'l'  -> MATCH
  Step 2: pop = 'e'  dequeue = 'e'  -> MATCH
  Step 3: pop = 'v'  dequeue = 'v'  -> MATCH
  Step 4: pop = 'e'  dequeue = 'e'  -> MATCH
  Step 5: pop = 'l'  dequeue = 'l'  -> MATCH
Result : level -> SYMMETRIC (Palindrome signature detected)
----------------------------------------
Ingesting packet : X9Y8Z7
  Step 1: pop = '7'  dequeue = 'X'  -> MISMATCH
  Step 2: pop = 'Z'  dequeue = '9'  -> MISMATCH
  Step 3: pop = '8'  dequeue = 'Y'  -> MISMATCH
```

```
  Step 4: pop = 'Y'  dequeue = '8'  -> MISMATCH
  Step 5: pop = '9'  dequeue = 'Z'  -> MISMATCH
  Step 6: pop = 'X'  dequeue = '7'  -> MISMATCH
Result : X9Y8Z7 -> NOT SYMMETRIC
----------------------------------------
Ingesting packet : racecar
  Step 1: pop = 'r'  dequeue = 'r'  -> MATCH
  Step 2: pop = 'a'  dequeue = 'a'  -> MATCH
  Step 3: pop = 'c'  dequeue = 'c'  -> MATCH
  Step 4: pop = 'e'  dequeue = 'e'  -> MATCH
  Step 5: pop = 'c'  dequeue = 'c'  -> MATCH
  Step 6: pop = 'a'  dequeue = 'a'  -> MATCH
  Step 7: pop = 'r'  dequeue = 'r'  -> MATCH
Result : racecar -> SYMMETRIC (Palindrome signature detected)
----------------------------------------
          AUDIT COMPLETE
========================================
```

Explanation

For "A1B2B1A" every popped character equals the dequeued character, so the packet is symmetric. For "X9Y8Z7" the first pair is 7 against X, which already mismatches, so the packet is not symmetric.

QUESTION NO: 3

Multithreaded Telemetry Engine Using Runnable and Thread

Build a multithreaded telemetry engine. LogProcessorTask implements Runnable and logs five iterations with Thread.sleep(1000). TelemetryThread extends Thread and logs five CPU and memory health updates every 800 ms. AsyncWebEngineApp prints the main thread name, creates both workers, assigns the custom names "Telemetry-Worker" and "Log-Worker", starts them simultaneously and logs their execution states.

File 1: LogProcessorTask.java

- **implements Runnable**: the task is separated from the thread that runs it.
- run() loops five times, printing the iteration with Thread.currentThread().getName(), then sleeps for 1000 ms.

Java Program:

J LogProcessorTask.java

```java
public class LogProcessorTask implements Runnable {

    @Override
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(
                Thread.currentThread().getName()
                + " - Log iteration: " + i
            );

            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }
    }
}
```

File 2: TelemetryThread.java

- **extends Thread**: overrides run() directly.
- Logs five system health updates (CPU cores and used memory) with an 800 ms pause between updates.

Java Program:

J TelemetryThread.java

```java
public class TelemetryThread extends Thread {

```

```
3         @Override
4         public void run() {
5             Runtime runtime = Runtime.getRuntime();
6
7             for (int i = 1; i <= 5; i++) {
8                 long usedMemoryMB = (runtime.totalMemory() - runtime.freeMemory())
9                                     / (1024 * 1024);
10                int cpuCores = runtime.availableProcessors();
11
12                System.out.println(
13                    Thread.currentThread().getName()
14                    + " - System health update: " + i
15                    + " | CPU cores: " + cpuCores
16                    + " | Memory used: " + usedMemoryMB + " MB"
17                );
18
19                try {
20                    Thread.sleep(800);
21                } catch (InterruptedException e) {
22                    Thread.currentThread().interrupt();
23                    return;
24                }
25            }
26        }
27    }
```

File 3: AsyncWebEngineApp.java

- Prints the default JVM thread name ("main").
- Creates TelemetryThread and a new Thread(logTask), assigns the custom names and calls start() on both.
- Logs thread states and uses join() so main waits for both workers before it terminates.

Main concept: Concurrency. Both workers run at the same time, so their output lines interleave.

Java Program:

J AsyncWebEngineApp.java

```
1     public class AsyncWebEngineApp {
2
3         public static void main(String[] args) throws InterruptedException {
4
5             System.out.println("Main thread: " + Thread.currentThread().getName());
6
7             // Thread created by extending Thread
8             TelemetryThread telemetry = new TelemetryThread();
9             telemetry.setName("Telemetry-Worker");
10
11            // Thread created by passing a Runnable into a new Thread object
12            LogProcessorTask logTask = new LogProcessorTask();
13            Thread logThread = new Thread(logTask);
14            logThread.setName("Log-Worker");
```

```
15
16          System.out.println("Telemetry-Worker state before start(): " + telemetry.getState());
17          System.out.println("Log-Worker state before start()      : " + logThread.getState());
18
19          // Start both threads so they run concurrently
20          telemetry.start();
21          logThread.start();
22
23          System.out.println("Active threads started.");
24          System.out.println("Telemetry-Worker alive: " + telemetry.isAlive());
25          System.out.println("Log-Worker alive      : " + logThread.isAlive());
26
27          // Main waits for both workers to finish before terminating
28          telemetry.join();
29          logThread.join();
30
31          System.out.println("Both worker threads completed. Main thread exiting.");
32      }
33  }
```

Program OutPut:

```
> java AsyncWebEngineApp
Main thread: main
Telemetry-Worker state before start(): NEW
Log-Worker state before start()      : NEW
Active threads started.
Log-Worker - Log iteration: 1
Telemetry-Worker alive: true
Log-Worker alive      : true
Telemetry-Worker - System health update: 1 | CPU cores: 1 | Memory used: 2 MB
Telemetry-Worker - System health update: 2 | CPU cores: 1 | Memory used: 3 MB
Log-Worker - Log iteration: 2
Telemetry-Worker - System health update: 3 | CPU cores: 1 | Memory used: 3 MB
Log-Worker - Log iteration: 3
Telemetry-Worker - System health update: 4 | CPU cores: 1 | Memory used: 3 MB
Log-Worker - Log iteration: 4
Telemetry-Worker - System health update: 5 | CPU cores: 1 | Memory used: 3 MB
Log-Worker - Log iteration: 5
Both worker threads completed. Main thread exiting.
```

Explanation

Calling start() creates a new thread of execution, whereas calling run() directly would execute on the main thread. The 800 ms and 1000 ms sleep intervals differ, which is why the telemetry and log lines interleave. The order of lines can change slightly from run to run because thread scheduling is controlled by the operating system.

QUESTION NO: 4

Parallel Portfolio Assessment Engine Using Fork/Join

Use the Fork/Join framework to assess a large portfolio. ParallelRiskCalculator extends RecursiveTask to sum the risk scores of an array (sequential when the segment is at most THRESHOLD = 1000, otherwise split into two sub-tasks with fork() and join()). PortfolioThresholdTask extends RecursiveAction to apply a 5% volatility stress-test penalty in place to every value above a risk cap. ForkJoinPortfolioApp generates a 10,000-element array, runs both tasks in a ForkJoinPool and reports processing times against serial computation.

File 1: ParallelRiskCalculator.java

- **extends RecursiveTask<Double>**: returns the cumulative risk score.
- If the segment size is at most 1000 it sums directly; otherwise it splits the range, calls fork() on one half, computes the other half and join()s the result.

Main concept: Divide and conquer with recursive decomposition.

Java Program:

J ParallelRiskCalculator.java

```java
import java.util.concurrent.RecursiveTask;

public class ParallelRiskCalculator extends RecursiveTask<Double> {

    private static final int THRESHOLD = 1000;

    private final double[] assetValues;
    private final int start;
    private final int end;

    public ParallelRiskCalculator(double[] assetValues, int start, int end) {
        this.assetValues = assetValues;
        this.start = start;
        this.end = end;
    }

    @Override
    protected Double compute() {
        int length = end - start;

        // Small segment: compute the sum sequentially
        if (length <= THRESHOLD) {
            double sum = 0;
            for (int i = start; i < end; i++) {
                sum += assetValues[i];
            }
            return sum;
        }

        // Large segment: split into two sub-tasks
```

```
            int mid = start + length / 2;
            ParallelRiskCalculator left  = new ParallelRiskCalculator(assetValues, start, mid);
            ParallelRiskCalculator right = new ParallelRiskCalculator(assetValues, mid, end);

            left.fork();                        // run left asynchronously
            double rightResult = right.compute(); // compute right in this thread
            double leftResult = left.join();    // wait for left result

            return leftResult + rightResult;
        }
    }
```

File 2: PortfolioThresholdTask.java

- **extends RecursiveAction**: returns no value and modifies the array in place.
- Every asset value above the risk cap is multiplied by 1.05 (5% stress-test penalty); large ranges are split recursively.

Java Program:

J PortfolioThresholdTask.java

```
import java.util.concurrent.RecursiveAction;

public class PortfolioThresholdTask extends RecursiveAction {

    private static final int THRESHOLD = 1000;
    private static final double STRESS_PENALTY = 1.05; // +5% volatility penalty

    private final double[] assetValues;
    private final int start;
    private final int end;
    private final double riskCap;

    public PortfolioThresholdTask(double[] assetValues, int start, int end, double riskCap) {
        this.assetValues = assetValues;
        this.start = start;
        this.end = end;
        this.riskCap = riskCap;
    }

    @Override
    protected void compute() {
        int length = end - start;

        if (length <= THRESHOLD) {
            // In-place adjustment of every value that exceeds the risk cap
            for (int i = start; i < end; i++) {
                if (assetValues[i] > riskCap) {
                    assetValues[i] = assetValues[i] * STRESS_PENALTY;
                }
            }
            return;
```

```
32            }
33
34            int mid = start + length / 2;
35            PortfolioThresholdTask left  = new PortfolioThresholdTask(assetValues, start, mid,
    riskCap);
36            PortfolioThresholdTask right = new PortfolioThresholdTask(assetValues, mid, end,
    riskCap);
37
38            left.fork();
39            right.compute();
40            left.join();
41        }
42    }
```

File 3: ForkJoinPortfolioApp.java

- Generates a 10,000-element synthetic asset array and creates a ForkJoinPool.
- Runs both tasks, verifies that the parallel result equals the serial result and prints timings.
- Also benchmarks a 20,000,000-element array so the effect of parallelism on large data is visible.

Java Program:

J ForkJoinPortfolioApp.java

```
1     import java.util.Random;
2     import java.util.concurrent.ForkJoinPool;
3
4     public class ForkJoinPortfolioApp {
5
6         static double serialSum(double[] data) {
7             double sum = 0;
8             for (double v : data) {
9                 sum += v;
10            }
11            return sum;
12        }
13
14        static double[] generate(int size, long seed) {
15            Random random = new Random(seed);
16            double[] data = new double[size];
17            for (int i = 0; i < size; i++) {
18                data[i] = 10 + random.nextDouble() * 990;   // 10.00 .. 1000.00
19            }
20            return data;
21        }
22
23        // Average time (ms) of serial vs parallel summation over 'runs' repetitions
24        static double[] timeBoth(double[] data, ForkJoinPool pool, int runs) {
25            double sink = 0;
26            long serialTotal = 0, parallelTotal = 0;
27            for (int r = 0; r < runs; r++) {
28                long t0 = System.nanoTime();
```

```
            sink += serialSum(data);
            serialTotal += System.nanoTime() - t0;

            long t1 = System.nanoTime();
            sink += pool.invoke(new ParallelRiskCalculator(data, 0, data.length));
            parallelTotal += System.nanoTime() - t1;
        }
        if (sink == 42) System.out.print("");   // keep JIT from discarding the work
        return new double[] { serialTotal / 1e6 / runs, parallelTotal / 1e6 / runs };
    }

    public static void main(String[] args) {

        final int SIZE = 10_000;
        final double RISK_CAP = 800.0;

        double[] assets = generate(SIZE, 247198L);
        ForkJoinPool pool = new ForkJoinPool();

        System.out.println("========================================");
        System.out.println("  FORK/JOIN PARALLEL PORTFOLIO ENGINE");
        System.out.println("========================================");
        System.out.println("Asset array size    : " + SIZE);
        System.out.println("Available cores     : " +
Runtime.getRuntime().availableProcessors());
        System.out.println("Sequential THRESHOLD: 1000");
        System.out.println("----------------------------------------");

        // ---- Task 1: cumulative risk score (RecursiveTask) ----
        long s0 = System.nanoTime();
        double serialRisk = serialSum(assets);
        long serialNs = System.nanoTime() - s0;

        long p0 = System.nanoTime();
        double parallelRisk = pool.invoke(new ParallelRiskCalculator(assets, 0, SIZE));
        long parallelNs = System.nanoTime() - p0;

        System.out.printf("Serial risk score   : %.2f%n", serialRisk);
        System.out.printf("Parallel risk score : %.2f%n", parallelRisk);
        System.out.println("Scores match        : " + (Math.abs(serialRisk - parallelRisk) < 1e-
6));
        System.out.printf("Single-run serial   : %.4f ms%n", serialNs / 1e6);
        System.out.printf("Single-run parallel : %.4f ms (includes pool start-up)%n", parallelNs
/ 1e6);
        System.out.println("----------------------------------------");

        // ---- Task 2: stress-test penalty (RecursiveAction) ----
        int before = 0;
        for (double v : assets) if (v > RISK_CAP) before++;

        long a0 = System.nanoTime();
        pool.invoke(new PortfolioThresholdTask(assets, 0, SIZE, RISK_CAP));
        long adjustNs = System.nanoTime() - a0;

        double stressedRisk = pool.invoke(new ParallelRiskCalculator(assets, 0, SIZE));
```

```
81
82          System.out.println("Risk cap            : " + RISK_CAP);
83          System.out.println("Assets above cap    : " + before);
84          System.out.printf("Stress-test time    : %.4f ms%n", adjustNs / 1e6);
85          System.out.printf("Risk after +5%% test : %.2f%n", stressedRisk);
86          System.out.printf("Risk increase       : %.2f (%.2f%%)%n",
87                  stressedRisk - parallelRisk, (stressedRisk - parallelRisk) / parallelRisk * 100);
88          System.out.println("----------------------------------------");
89
90          // ---- Warmed-up benchmark, 10,000 elements ----
91          timeBoth(assets, pool, 200);                        // JIT warm-up
92          double[] t = timeBoth(assets, pool, 2000);
93          System.out.println("Benchmark (10,000 elements, average of 2000 runs after warm-up)");
94          System.out.printf("  Serial   : %.5f ms%n", t[0]);
95          System.out.printf("  Parallel : %.5f ms%n", t[1]);
96          System.out.printf("  Speedup  : %.2fx%n", t[0] / t[1]);
97          System.out.println("----------------------------------------");
98
99          // ---- Larger dataset to show real parallel speedup ----
100         double[] big = generate(20_000_000, 1L);
101         timeBoth(big, pool, 3);                             // warm-up
102         double[] tb = timeBoth(big, pool, 10);
103         System.out.println("Benchmark (20,000,000 elements, average of 10 runs)");
104         System.out.printf("  Serial   : %.3f ms%n", tb[0]);
105         System.out.printf("  Parallel : %.3f ms%n", tb[1]);
106         System.out.printf("  Speedup  : %.2fx%n", tb[0] / tb[1]);
107         System.out.println("========================================");
108
109         pool.shutdown();
110     }
111 }
```

Program OutPut:

```
> java ForkJoinPortfolioApp
========================================
  FORK/JOIN PARALLEL PORTFOLIO ENGINE
========================================
Asset array size    : 10000
Available cores     : 1
Sequential THRESHOLD: 1000
----------------------------------------
Serial risk score   : 4993155.46
Parallel risk score : 4993155.46
Scores match        : true
Single-run serial   : 0.1236 ms
Single-run parallel : 1.7062 ms (includes pool start-up)
----------------------------------------
Risk cap            : 800.0
Assets above cap    : 1980
Stress-test time    : 0.7064 ms
Risk after +5% test : 5082302.86
Risk increase       : 89147.40 (1.79%)
----------------------------------------
Benchmark (10,000 elements, average of 2000 runs after warm-up)
  Serial   : 0.01523 ms
  Parallel : 0.04335 ms
  Speedup  : 0.35x
----------------------------------------
Benchmark (20,000,000 elements, average of 10 runs)
  Serial   : 28.265 ms
  Parallel : 33.010 ms
  Speedup  : 0.86x
========================================
```

Analysis of Timing Results

The parallel and serial risk scores are identical, which confirms the decomposition is correct. Fork/Join speedup depends on the amount of work per task and on the number of CPU cores: for only 10,000 elements the work is so small that task creation and scheduling overhead can outweigh the gain, so the benchmark is repeated on a much larger array where each sub-task has enough work to benefit from running on several cores. The "Available cores" line in the output shows how many cores the run could use.

QUESTION NO: 5

Parallel IoT Sensor Search Engine Using an Abstract Class

Construct an extensible parallel search engine. SensorSearchEngine is an abstract parent class with protected sensorData[], startIdx and endIdx fields and an abstract processGrid() method. ParallelMinSearch finds the minimum temperature and ParallelMaxSearch finds the peak vibration spike using multithreaded searches. SensorGridMonitorApp partitions a 50,000-element array across four worker threads, runs the min and max reductions simultaneously and logs the global values and execution timing.

File 1: SensorSearchEngine.java

- Abstract class with protected sensorData (double[]), startIdx and endIdx.
- **abstract processGrid()**: every concrete engine must implement its own search logic.

Main concept: Abstraction and extensibility: new search engines can be added without changing the monitor.

Java Program:

J SensorSearchEngine.java

```java
public abstract class SensorSearchEngine {

    protected double[] sensorData;
    protected int startIdx;
    protected int endIdx;      // exclusive

    public SensorSearchEngine(double[] sensorData, int startIdx, int endIdx) {
        this.sensorData = sensorData;
        this.startIdx = startIdx;
        this.endIdx = endIdx;
    }

    public abstract void processGrid() throws InterruptedException;
}
```

File 2: ParallelMinSearch.java

- Overrides processGrid(): splits the range into four segments, one worker thread per segment.
- Each worker finds a local minimum; after join() the local results are reduced to the global minimum.

Java Program:

J ParallelMinSearch.java

```java
public class ParallelMinSearch extends SensorSearchEngine {

```

```
3         private final int workerCount;
4         private double globalMin = Double.MAX_VALUE;
5
6         public ParallelMinSearch(double[] sensorData, int startIdx, int endIdx, int workerCount) {
7             super(sensorData, startIdx, endIdx);
8             this.workerCount = workerCount;
9         }
10
11        @Override
12        public void processGrid() throws InterruptedException {
13            double[] localMin = new double[workerCount];
14            Thread[] workers = new Thread[workerCount];
15            int total = endIdx - startIdx;
16            int chunk = total / workerCount;
17
18            for (int w = 0; w < workerCount; w++) {
19                final int id = w;
20                final int from = startIdx + w * chunk;
21                final int to = (w == workerCount - 1) ? endIdx : from + chunk;
22
23                workers[w] = new Thread(() -> {
24                    double min = Double.MAX_VALUE;
25                    for (int i = from; i < to; i++) {
26                        if (sensorData[i] < min) {
27                            min = sensorData[i];
28                        }
29                    }
30                    localMin[id] = min;
31                    System.out.printf("[MIN-Worker-%d] segment [%d..%d) local minimum = %.2f%n",
32                            id + 1, from, to, min);
33                }, "MIN-Worker-" + (w + 1));
34                workers[w].start();
35            }
36
37            for (Thread t : workers) {
38                t.join();
39            }
40
41            // Reduction of the partial results
42            for (double m : localMin) {
43                if (m < globalMin) {
44                    globalMin = m;
45                }
46            }
47        }
48
49        public double getGlobalMin() {
50            return globalMin;
51        }
52    }
```

File 3: ParallelMaxSearch.java

- Overrides processGrid() in the same way to locate the peak value in each segment.

- ❖ The four local peaks are reduced to the global maximum.

Java Program:

J ParallelMaxSearch.java

```java
public class ParallelMaxSearch extends SensorSearchEngine {

    private final int workerCount;
    private double globalMax = -Double.MAX_VALUE;

    public ParallelMaxSearch(double[] sensorData, int startIdx, int endIdx, int workerCount) {
        super(sensorData, startIdx, endIdx);
        this.workerCount = workerCount;
    }

    @Override
    public void processGrid() throws InterruptedException {
        double[] localMax = new double[workerCount];
        Thread[] workers = new Thread[workerCount];
        int total = endIdx - startIdx;
        int chunk = total / workerCount;

        for (int w = 0; w < workerCount; w++) {
            final int id = w;
            final int from = startIdx + w * chunk;
            final int to = (w == workerCount - 1) ? endIdx : from + chunk;

            workers[w] = new Thread(() -> {
                double max = -Double.MAX_VALUE;
                for (int i = from; i < to; i++) {
                    if (sensorData[i] > max) {
                        max = sensorData[i];
                    }
                }
                localMax[id] = max;
                System.out.printf("[MAX-Worker-%d] segment [%d..%d) local peak    = %.2f%n",
                        id + 1, from, to, max);
            }, "MAX-Worker-" + (w + 1));
            workers[w].start();
        }

        for (Thread t : workers) {
            t.join();
        }

        for (double m : localMax) {
            if (m > globalMax) {
                globalMax = m;
            }
        }
    }

    public double getGlobalMax() {
        return globalMax;
    }
```

```
51    }
```

File 4: SensorGridMonitorApp.java

- ❖ Builds a 50,000-element telemetry array and plants a known minimum and maximum for verification.
- ❖ Runs the min engine and max engine at the same time on separate threads, using four workers each.
- ❖ Prints the global critical values, verifies them against a serial scan and logs the timing.

Main concept: Parallel reduction: partial results from independent segments are combined into one global result.

Java Program:

J SensorGridMonitorApp.java

```java
import java.util.Random;

public class SensorGridMonitorApp {

    public static void main(String[] args) throws InterruptedException {

        final int SIZE = 50_000;
        final int WORKERS = 4;

        // Synthetic telemetry readings between 20.0 and 120.0
        Random random = new Random(247198L);
        double[] sensorData = new double[SIZE];
        for (int i = 0; i < SIZE; i++) {
            sensorData[i] = 20 + random.nextDouble() * 100;
        }

        // Plant known extremes so the result can be verified
        sensorData[12_345] = 3.75;     // critical low temperature
        sensorData[41_999] = 187.60;   // peak vibration spike

        ParallelMinSearch minSearch = new ParallelMinSearch(sensorData, 0, SIZE, WORKERS);
        ParallelMaxSearch maxSearch = new ParallelMaxSearch(sensorData, 0, SIZE, WORKERS);

        System.out.println("========================================");
        System.out.println("     IoT SENSOR GRID MONITOR");
        System.out.println("========================================");
        System.out.println("Sensor readings : " + SIZE);
        System.out.println("Workers per scan: " + WORKERS);
        System.out.println("----------------------------------------");

        // Run the MIN and MAX reductions simultaneously
        Thread minRunner = new Thread(() -> {
            try { minSearch.processGrid(); } catch (InterruptedException e) { }
        }, "Min-Engine");
        Thread maxRunner = new Thread(() -> {
```

```
36              try { maxSearch.processGrid(); } catch (InterruptedException e) { }
37          }, "Max-Engine");
38
39          long start = System.nanoTime();
40          minRunner.start();
41          maxRunner.start();
42          minRunner.join();
43          maxRunner.join();
44          long parallelNs = System.nanoTime() - start;
45
46          System.out.println("----------------------------------------");
47          System.out.printf("Global critical MIN temperature : %.2f%n", minSearch.getGlobalMin());
48          System.out.printf("Global critical MAX vibration   : %.2f%n", maxSearch.getGlobalMax());
49          System.out.println("----------------------------------------");
50
51          // Serial check for correctness and timing comparison
52          long s0 = System.nanoTime();
53          double sMin = Double.MAX_VALUE, sMax = -Double.MAX_VALUE;
54          for (double v : sensorData) {
55              if (v < sMin) sMin = v;
56              if (v > sMax) sMax = v;
57          }
58          long serialNs = System.nanoTime() - s0;
59
60          System.out.println("Verification vs serial scan     : "
61                  + ((sMin == minSearch.getGlobalMin() && sMax == maxSearch.getGlobalMax())
62                          ? "PASSED" : "FAILED"));
63          System.out.printf("Parallel execution time : %.3f ms%n", parallelNs / 1e6);
64          System.out.printf("Serial execution time   : %.3f ms%n", serialNs / 1e6);
65          System.out.println("(Thread start-up cost dominates for only 50,000 elements)");
66          System.out.println("========================================");
67      }
68  }
```

Program OutPut:

```
> java SensorGridMonitorApp
========================================
     IoT SENSOR GRID MONITOR
========================================
Sensor readings : 50000
Workers per scan: 4
----------------------------------------
[MIN-Worker-1] segment [0..12500) local minimum = 3.75
[MIN-Worker-2] segment [12500..25000) local minimum = 20.02
[MIN-Worker-4] segment [37500..50000) local minimum = 20.00
[MIN-Worker-3] segment [25000..37500) local minimum = 20.02
[MAX-Worker-1] segment [0..12500) local peak    = 120.00
[MAX-Worker-3] segment [25000..37500) local peak    = 119.99
[MAX-Worker-2] segment [12500..25000) local peak    = 120.00
[MAX-Worker-4] segment [37500..50000) local peak    = 187.60
----------------------------------------
Global critical MIN temperature : 3.75
Global critical MAX vibration   : 187.60
----------------------------------------
Verification vs serial scan     : PASSED
Parallel execution time : 58.640 ms
Serial execution time   : 1.445 ms
(Thread start-up cost dominates for only 50,000 elements)
========================================
```

Explanation

The planted values (3.75 in segment 1 and 187.60 in segment 4) are correctly found as the global minimum and maximum. For a small array of 50,000 elements, the cost of creating threads is larger than the cost of scanning, so the parallel version is not faster than the serial scan here; parallel reductions pay off for much larger datasets.

QUESTION NO: 6

Inventory Catalog Concurrency Benchmark

Build a benchmark in InventoryCatalogManager.java that maintains a shared product stock catalog (String productId, Integer stockCount). benchmarkSynchronizedMap() wraps a HashMap with Collections.synchronizedMap() and benchmarkConcurrentMap() uses ConcurrentHashMap. Each method spawns a pool of 10 worker threads performing 50,000 stock lookup and update operations in total, then prints execution time and throughput (operations/second).

File 1: InventoryCatalogManager.java

- Creates a catalog of 1,000 products, each with an initial stock count.
- **benchmarkSynchronizedMap()**: coarse-grained locking, one lock for the whole map.
- **benchmarkConcurrentMap()**: ConcurrentHashMap with fine-grained bucket-level locking.
- Each benchmark uses a fixed pool of 10 threads performing 50,000 lookups and updates, measures time with System.nanoTime() and checks that no update was lost.

Main concept: Lock contention. A synchronized map allows one thread at a time, whereas ConcurrentHashMap lets threads working on different buckets proceed in parallel.

Java Program:

J InventoryCatalogManager.java

```java
import java.util.Collections;
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicLong;

public class InventoryCatalogManager {

    private static final int PRODUCTS       = 1000;
    private static final int INITIAL_STOCK  = 1_000_000;
    private static final int WORKER_THREADS = 10;
    private static final int TOTAL_OPS      = 50_000;
    private static final int OPS_PER_THREAD = TOTAL_OPS / WORKER_THREADS;

    // Populate a catalog: productId -> stockCount
    private static void fill(Map<String, Integer> catalog) {
        for (int i = 0; i < PRODUCTS; i++) {
            catalog.put("PROD-" + i, INITIAL_STOCK);
        }
    }

    // Runs 10 workers; each does lookups and stock updates. Returns elapsed nanoseconds.
    private static long runWorkload(Map<String, Integer> catalog, AtomicLong updates)
            throws InterruptedException {
```

```
28
29          ExecutorService pool = Executors.newFixedThreadPool(WORKER_THREADS);
30          long start = System.nanoTime();
31
32          for (int t = 0; t < WORKER_THREADS; t++) {
33              final int seed = t;
34              pool.submit(() -> {
35                  java.util.Random rnd = new java.util.Random(seed);
36                  for (int op = 0; op < OPS_PER_THREAD; op++) {
37                      String id = "PROD-" + rnd.nextInt(PRODUCTS);
38                      if (op % 2 == 0) {
39                          catalog.get(id);                        // stock lookup
40                      } else {
41                          catalog.merge(id, -1, Integer::sum);    // atomic stock update
42                          updates.incrementAndGet();
43                      }
44                  }
45              });
46          }
47
48          pool.shutdown();
49          pool.awaitTermination(1, TimeUnit.MINUTES);
50          return System.nanoTime() - start;
51      }
52
53      private static long totalStock(Map<String, Integer> catalog) {
54          long sum = 0;
55          for (int v : catalog.values()) sum += v;
56          return sum;
57      }
58
59      private static void report(String name, Map<String, Integer> catalog,
60                                 long nanos, long updates) {
61          double ms = nanos / 1e6;
62          double throughput = TOTAL_OPS / (nanos / 1e9);
63          long expected = (long) PRODUCTS * INITIAL_STOCK - updates;
64          System.out.println("Collection     : " + name);
65          System.out.printf("Execution time : %.3f ms%n", ms);
66          System.out.printf("Throughput     : %,.0f operations/second%n", throughput);
67          System.out.println("Data integrity : "
68                  + (totalStock(catalog) == expected ? "PASSED (no lost updates)" : "FAILED"));
69      }
70
71      public static void benchmarkSynchronizedMap() throws InterruptedException {
72          Map<String, Integer> catalog = Collections.synchronizedMap(new HashMap<>());
73          fill(catalog);
74          AtomicLong updates = new AtomicLong();
75          long nanos = runWorkload(catalog, updates);
76          report("Collections.synchronizedMap(HashMap)", catalog, nanos, updates.get());
77      }
78
79      public static void benchmarkConcurrentMap() throws InterruptedException {
80          Map<String, Integer> catalog = new ConcurrentHashMap<>();
81          fill(catalog);
82          AtomicLong updates = new AtomicLong();
```

```
83            long nanos = runWorkload(catalog, updates);
84            report("ConcurrentHashMap", catalog, nanos, updates.get());
85        }
86
87        public static void main(String[] args) throws InterruptedException {
88
89            // Warm-up so the JIT compiler does not distort the first measurement
90            for (int i = 0; i < 5; i++) {
91                runWorkload(Collections.synchronizedMap(prefilled()), new AtomicLong());
92                runWorkload(prefilled(new ConcurrentHashMap<>()), new AtomicLong());
93            }
94
95            System.out.println("=========================================");
96            System.out.println("  INVENTORY CATALOG CONCURRENCY BENCHMARK");
97            System.out.println("=========================================");
98            System.out.println("Products       : " + PRODUCTS);
99            System.out.println("Worker threads : " + WORKER_THREADS);
100           System.out.println("Total ops      : " + TOTAL_OPS + " (lookups + updates)");
101           System.out.println("-----------------------------------------");
102
103           benchmarkSynchronizedMap();
104           System.out.println("-----------------------------------------");
105           benchmarkConcurrentMap();
106           System.out.println("=========================================");
107       }
108
109       private static Map<String, Integer> prefilled() {
110           return prefilled(new HashMap<>());
111       }
112
113       private static Map<String, Integer> prefilled(Map<String, Integer> map) {
114           fill(map);
115           return map;
116       }
117   }
```

Program OutPut:

```
> java InventoryCatalogManager
========================================
  INVENTORY CATALOG CONCURRENCY BENCHMARK
========================================
Products       : 1000
Worker threads : 10
Total ops      : 50000 (lookups + updates)
----------------------------------------
Collection     : Collections.synchronizedMap(HashMap)
Execution time : 14.362 ms
Throughput     : 3,481,460 operations/second
Data integrity : PASSED (no lost updates)
----------------------------------------
Collection     : ConcurrentHashMap
Execution time : 11.818 ms
Throughput     : 4,230,926 operations/second
Data integrity : PASSED (no lost updates)
========================================
```

Explanation

Collections.synchronizedMap() guards every operation with one mutex, so all 10 threads queue for the same lock. ConcurrentHashMap reads without locking and locks only the bucket being updated, so threads touching different products do not block each other. The benchmark is warmed up first so JIT compilation does not distort the first measurement, and the data-integrity check proves that both maps handled all updates correctly. The throughput gap grows as the number of cores and threads increases.

QUESTION NO: 7

Cache Locality Benchmark: Row-Major vs Column-Major Traversal

In CacheLocalityBenchmark.java, create a 2000 × 2000 double matrix. traverseRowMajor() sums all elements with the loops for i, for j, and traverseColumnMajor() uses the swapped loops for j, for i. Measure both with System.nanoTime() over 10 repeated iterations, print the times in milliseconds, compute the speedup ratio of row-major over column-major traversal, and explain the results using CPU cache architecture and spatial locality.

File 1: CacheLocalityBenchmark.java

- **matrix[2000][2000]**: double-precision matrix filled with values.
- **traverseRowMajor()**: inner loop moves along a row (contiguous memory).
- **traverseColumnMajor()**: inner loop moves down a column (a different row array on every step).
- main() warms up the JIT, runs 10 timed iterations, prints each time in milliseconds, the averages and the speedup ratio.

Main concept: Spatial locality and cache lines.

Java Program:

J CacheLocalityBenchmark.java

```java
public class CacheLocalityBenchmark {

    private static final int N = 2000;
    private static final int ITERATIONS = 10;

    private static double[][] matrix = new double[N][N];

    // Row-major: inner loop walks along a row (contiguous memory, stride 1)
    public static double traverseRowMajor() {
        double sum = 0;
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                sum += matrix[i][j];
            }
        }
        return sum;
    }

    // Column-major: inner loop jumps down a column (a different row array every step)
    public static double traverseColumnMajor() {
        double sum = 0;
        for (int j = 0; j < N; j++) {
            for (int i = 0; i < N; i++) {
                sum += matrix[i][j];
            }
        }
        return sum;
```

```
    }

    public static void main(String[] args) {

        // Fill the 2000 x 2000 matrix
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                matrix[i][j] = (i * 31 + j * 17) % 100 / 10.0;
            }
        }

        // JIT warm-up (not timed)
        double checksum = 0;
        for (int w = 0; w < 3; w++) {
            checksum += traverseRowMajor();
            checksum += traverseColumnMajor();
        }

        double[] rowTimes = new double[ITERATIONS];
        double[] colTimes = new double[ITERATIONS];
        double rowTotal = 0, colTotal = 0;

        System.out.println("========================================");
        System.out.println("     CACHE LOCALITY BENCHMARK (2000 x 2000)");
        System.out.println("========================================");
        System.out.println("Iteration | Row-Major (ms) | Column-Major (ms)");
        System.out.println("----------------------------------------");

        for (int it = 0; it < ITERATIONS; it++) {
            long t0 = System.nanoTime();
            double r = traverseRowMajor();
            long t1 = System.nanoTime();
            double c = traverseColumnMajor();
            long t2 = System.nanoTime();

            checksum += r + c;
            rowTimes[it] = (t1 - t0) / 1e6;
            colTimes[it] = (t2 - t1) / 1e6;
            rowTotal += rowTimes[it];
            colTotal += colTimes[it];

            System.out.printf("%9d | %14.3f | %17.3f%n", it + 1, rowTimes[it], colTimes[it]);
        }

        double rowAvg = rowTotal / ITERATIONS;
        double colAvg = colTotal / ITERATIONS;

        System.out.println("----------------------------------------");
        System.out.printf("Average Row-Major time    : %.3f ms%n", rowAvg);
        System.out.printf("Average Column-Major time : %.3f ms%n", colAvg);
        System.out.printf("Speedup (Column / Row)    : %.2fx%n", colAvg / rowAvg);
        System.out.println("Checksum (prevents dead-code removal): " + checksum);
        System.out.println("========================================");
    }
}
```

Program OutPut:

```
> java CacheLocalityBenchmark
========================================
     CACHE LOCALITY BENCHMARK (2000 x 2000)
========================================
Iteration | Row-Major (ms) | Column-Major (ms)
----------------------------------------
        1 |          5.899 |            45.097
        2 |          6.304 |            45.855
        3 |          5.727 |            44.055
        4 |          5.794 |            43.778
        5 |          5.797 |            46.946
        6 |          8.375 |            49.361
        7 |          5.665 |            45.292
        8 |          5.684 |            44.392
        9 |          6.125 |            44.177
       10 |          5.690 |            44.335
----------------------------------------
Average Row-Major time    : 6.106 ms
Average Column-Major time : 45.329 ms
Speedup (Column / Row)    : 7.42x
Checksum (prevents dead-code removal): 5.148E8
========================================
```

Execution Report: How Cache Architecture Explains the Results

Modern CPUs do not fetch single bytes from RAM; they load a whole cache line (typically 64 bytes) into the Level 1 and Level 2 caches. A double is 8 bytes, so one cache line holds 8 consecutive doubles.

Row-major traversal reads matrix[i][0], matrix[i][1], matrix[i][2] and so on, which are neighbours in memory (stride 1). One cache miss loads a line that serves the next 7 reads as cache hits, and the hardware prefetcher can detect the sequential pattern and load the next lines in advance. This gives a high cache hit rate.

Column-major traversal reads matrix[0][j], matrix[1][j], matrix[2][j] and so on. In Java each row is a separate array object, so consecutive reads are about 16 KB apart (2000 doubles per row) and each read touches a different cache line. The line is usually evicted before its other 7 elements are used on the next column pass, so nearly every access is a cache miss, the prefetcher cannot predict the jumping pattern, and the CPU stalls waiting for main memory (and for TLB lookups across many memory pages).

Both loops perform exactly the same number of additions, so the difference comes only from memory access order. The measured speedup ratio (column-major time divided by row-major time) is therefore well above 1, in line with the theoretical figure of roughly 8 accesses served per cache line. The exact ratio depends on the processor, its cache sizes and memory speed.